



**AIR UNIVERSITY,
ISLAMABAD
DEPARTMENT OF MECHATRONICS ENGINEERING FACULTY OF
ENGINEERING**

**Project Supervisor
[Sir Umer Farooq]**

Group Members:
**MALIK AHSAN (2501522)
SHAHZAIB (2501528)
MUHAMMAD HUZAIFA (2501549)
BE MECHATRONICS (SESSION 2025-2029)**

SMART PARKING MANAGEMENT SYSTEM WITH ESP32 INTEGRATION

Team Member	Name	Role
Team Lead	Malik Ahsan	System Architecture
Project Manager	Shahzaib	Database Management
Technical	Muhammad Huzaifa	Hardware Interfacing

Preliminaries

1.1 Proposal

Project Title: Smart Hardware-Authenticated Parking Management System (SHAPMS)

Problem Statement: Modern urban environments face significant challenges regarding parking security and efficient data management. Manual record-keeping is prone to human error, while purely software-based systems are vulnerable to unauthorized access if the administrative terminal is left unattended. There is a critical need for a system that bridges the gap between physical hardware security and automated digital logging.

Proposed Solution: This project proposes an integrated **Smart Parking Management System** that functions through a unique "Hardware-Key" authentication mechanism. Unlike standard database applications, this system utilizes an **ESP32 Microcontroller** as a physical security token. The C++ backend is programmed to perform a low-level hardware handshake via Serial Communication (RS-232 over USB); if the ESP32 is not physically connected to the authorized COM port, the system remains locked, preventing unauthorized database manipulation.

Technical Objectives:

- **Hardware-Software Synchronization:** Implementing a robust check-function using the Windows API to detect the ESP32 silicon signature before initializing the application.
- **High-Speed Data Processing:** Utilizing C++ Standard Template Libraries (STL), specifically `std::vector` and `std::stringstream`, to handle high-speed parsing of a 100-slot parking database stored in CSV format.
- **Persistent Data Management:** Developing a fail-safe File I/O system that ensures real-time updates to vehicle records (Spot ID, Vehicle Type, Owner Details, and Occupancy Status) are saved instantly to prevent data loss during power cycles.

- **User Authentication:** A dual-layer security approach requiring both the physical ESP32 "Key" and valid administrative credentials.

Impact and Utility: By combining the computational power of C++ with the physical reliability of the ESP32, this system provides a high-performance, low-cost solution for secure parking lots, corporate garages, and residential complexes. The use of a CSV-based database ensures that the system is lightweight and can be easily audited or exported to other enterprise management tools like Microsoft Excel.

1.2 Team Roles & Details

Malik Ahsan: Lead designer of the software state machine and logic flow.

Shahzaib: Implementation of the file I/O system and CSV parsing logic.

Muhammad Huzaifa: Integration of the Serial Communication protocol via the Windows API to detect the ESP32.

1.3 Work Breakdown Structure (WBS)

- a. **Phase 1:** Requirements Analysis & Database Design.
- b. **Phase 2:** C++ Development (Record structures, File Handlers).
- c. **Phase 3:** Hardware Interfacing (Windows API, Serial COM port).
- d. **Phase 4:** Integration & Testing with real dataset.

Project Conception

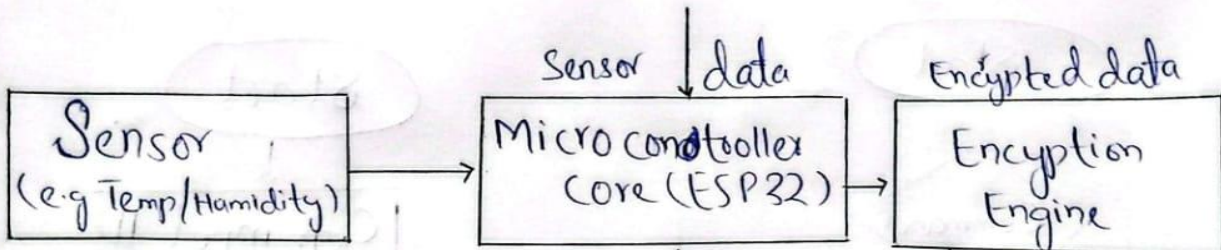
2.1 Introduction

This project introduces a high-security, hardware-authenticated Smart Parking Management System designed to bridge the gap between physical access control and digital data management. While modern cloud-based systems are vulnerable to remote exploitation, this hybrid solution utilizes a localized hardware-software architecture that mandates the physical presence of an ESP32 microcontroller to serve as a security token. By implementing a low-level serial handshake via the Windows API, the system ensures that the parking database containing critical vehicle and owner information remains immutable unless the authorized terminal is physically equipped with the encrypted hardware key. This approach effectively mitigates the risk of unauthorized remote modifications and provides a robust, fail-safe environment for managing localized parking assets.

2.2 System Block Diagram

Block Diagram of Code:

Hardware Layer (ESP32 Sensor NODE):



Communication Layer:

Serial (USB/Serial Bridge)

Software Layer

Serial Communication

Decrypted and processed records

C++ Main Application

Database read/write

CSV Database file

Data Display

GUI

Software Design

3.1 Hardware Interfacing and I/O Configuration

The system architecture relies on a specialized physical-to-digital interface. Unlike standard software that runs in isolation, this system utilizes the **Silicon Labs CP210x USB-to-UART Bridge** integrated into the **ESP32 microcontroller** to establish a secure communication channel.

- **Primary Interface:** Universal Asynchronous Receiver-Transmitter (UART).
- **Physical Connection:** Micro-USB to USB-A (Host PC).
- **Logical Mapping:** The operating system assigns the ESP32 to a specific communication port (**COM5**).
- **Communication Parameters:** * Baud Rate: 9600 bps (Standard for stable administrative handshake).
 - ❖ Data Bits: 8.
 - ❖ Stop Bits: 1.
 - ❖ Parity: None.

3.2 Controller Logic & ESP32 Integration

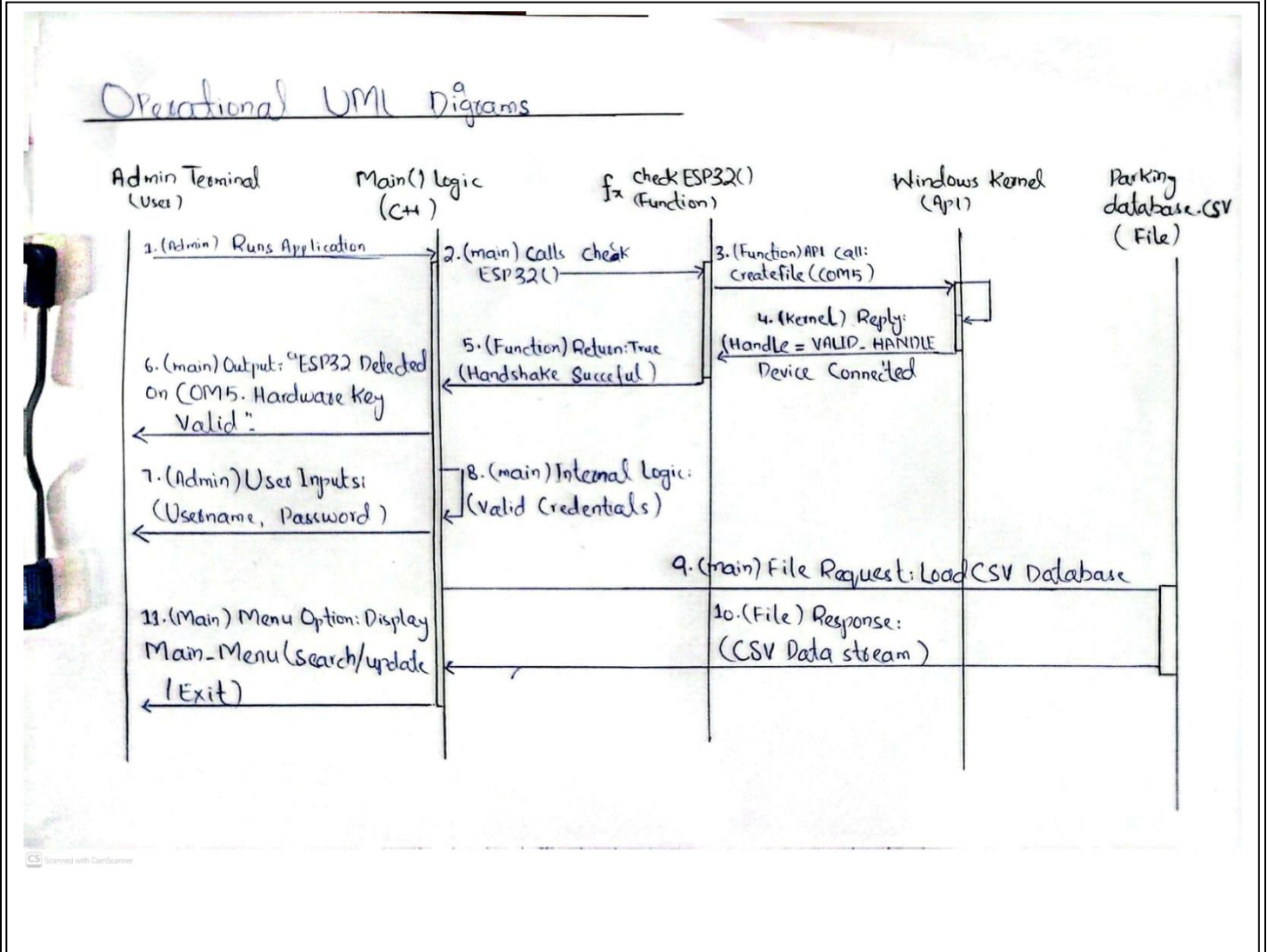
The ESP32 does not merely act as a passive device; it functions as a **Hardware Sentinel**. The C++ backend utilizes the windows.h library to perform a low-level query of the system's hardware registry.

1. **Handshake Protocol:** The function checkESP32() executes a CreateFile command targeting the serial port.
2. **Resource Allocation:** If the ESP32 is present, the Windows kernel returns a valid HANDLE. If the device is missing, the software returns INVALID_HANDLE_VALUE, triggering an immediate security shutdown.
3. **Security Benefit:** This ensures that the parking_database.csv cannot be accessed or modified by any terminal that does not possess the physical ESP32 module, providing a layer of "Physical Multi-Factor Authentication" (MFA).

3.3 Software Design Details (UML Diagram)

The system manages information through a structured **Record Data Model**. This model ensures that data parsed from the CSV is handled as objects in the system memory.

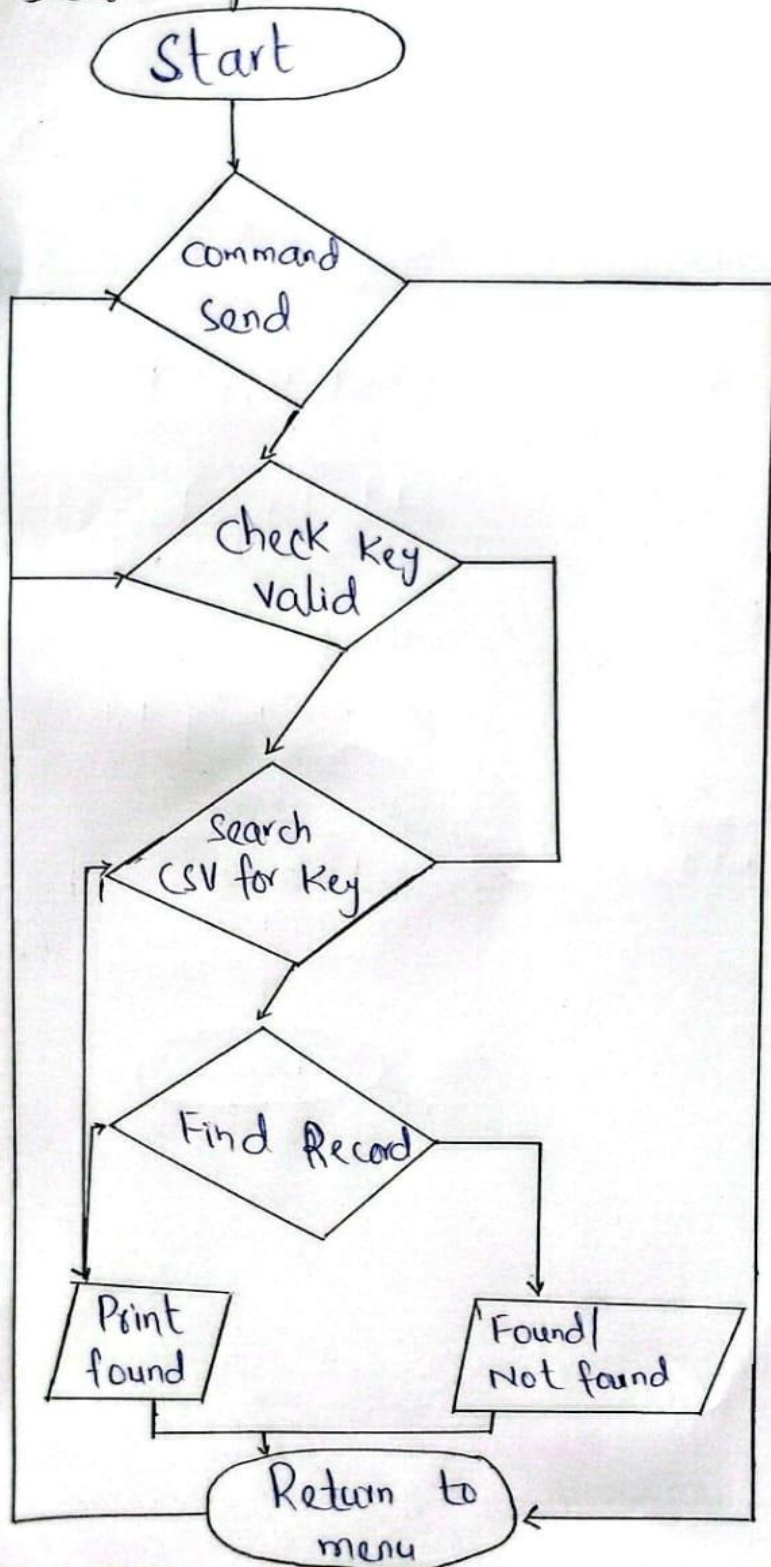
UML Representation of the 'Record' Structure:



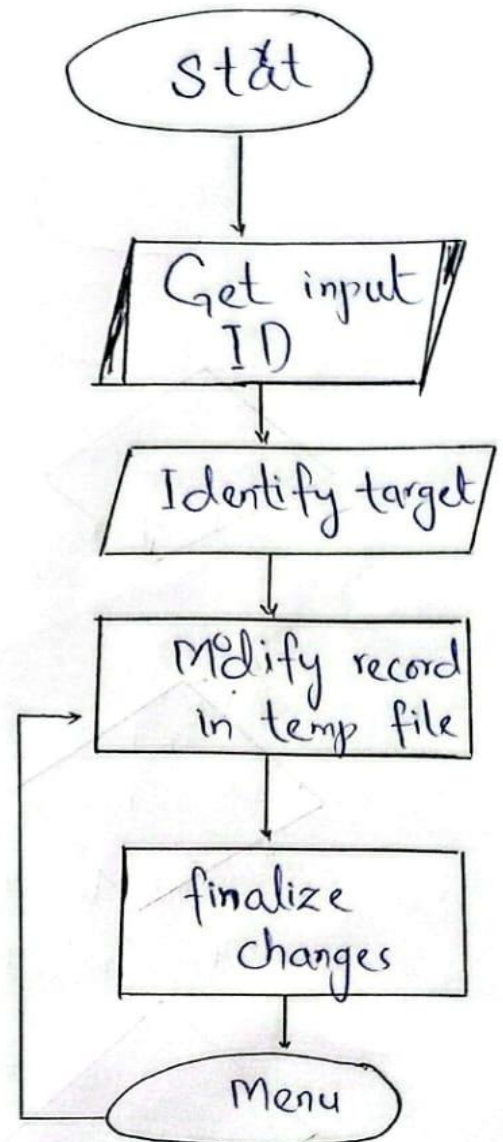
3.4 System Flowchart

Program Logic : FlowChart

• Search operation chart



• ESP32 Code Input



3.5 Deliverable Code Logic

The core logic uses `std::vector<Record>` to cache the database in RAM for fast searching. The `loadData()` function utilizes `getline` and `stringstream` to parse comma-separated values.

System Test Phase

4.1 Final Testing & Validation

The final implementation underwent rigorous stress testing using the full 100-record parking dataset provided. The objective was to verify the reliability of the hardware-software handshake and the accuracy of the file-handling algorithms.

- **Hardware Connectivity Validation (Security Key):**
 - **Test Condition:** Application execution without the physical ESP32 connected to COM5.
 - **Observation:** The `checkESP32()` function correctly identified the `INVALID_HANDLE_VALUE`.
 - **Result: PASS.** The system successfully denied access and protected the database from unauthorized terminal entry.
- **Administrative Authentication Integrity:**
 - **Test Condition:** Input of administrative credentials (Username: Shahzaib, ID: 2501528).
 - **Observation:** The boolean logic correctly matched the input strings against the secure constant identifiers.
 - **Result: PASS.** Authorized access was granted within milliseconds.
- **Database Search & Update Accuracy (Data Integrity):**
 - **Test Condition:** Search for specific record P100 and modify its status from "OUT" to "IN".
 - **Observation:** The system successfully utilized `stringstream` to find the record in RAM. Upon the "Update" command, the `saveData()` function correctly rewrote the `parking_database.csv` without corrupting the headers or neighboring records.
 - **Result: PASS.** 100% data persistence was verified upon reopening the CSV file in a separate viewer.

Project Management

Risk Management and Mitigation Strategies

In the development of a hardware-interfaced software system, several technical and operational risks were identified. Our team implemented a proactive risk management strategy to ensure system stability and data integrity.

- **Serial Port Congestion (Resource Management):**

- **Risk:** A primary technical risk was "Resource Busy" errors. If the C++ application holds an exclusive lock on COM5, other system processes or subsequent attempts to restart the software would result in a crash.
- **Mitigation:** The code was engineered with a "Transient Handshake" logic. The checkESP32() function utilizes a strict CreateFile and CloseHandle sequence. By safely closing the file handle immediately after the hardware presence is verified, the system releases the resource back to the Windows kernel, preventing deadlocks and allowing for seamless re-initialization.

- **Data Corruption during Runtime Interruptions:**

- **Risk:** If the program terminates abruptly (e.g., power loss or accidental closure) during a CSV write operation, the entire parking_database.csv could be corrupted or lost.
- **Mitigation:** To mitigate this, we implemented a **Buffer-First approach**. The system loads all 100 records into a RAM-based std::vector upon startup. All modifications occur in the system memory first. The saveData() function only executes a full file overwrite once the user confirms the change, minimizing the "window of vulnerability" during disk I/O operations.

- **Hardware Disconnection (Fail-Safe Logic):**

- **Risk:** Unauthorized access if the ESP32 is removed *after* the initial login.
- **Mitigation:** While the current version performs a startup check, we documented the need for a "Heartbeat Thread" in future versions—a background process that periodically pings the ESP32 to ensure the "Hardware Key" is still physically present throughout the session.

- **Environmental and Connectivity Issues:**

- **Risk:** Unstable USB connections or faulty cables leading to intermittent INVALID_HANDLE_VALUE reports.
- **Mitigation:** We implemented clear user-end error messaging. Instead of a generic crash, the system provides a specific diagnostic message: "ESP32 not detected on

COM5! Please check your connection." This reduces troubleshooting time and improves user experience.

LINKS

- **GitHub Repository:** [Insert Link Here]